

AI335L — DEEP LEARNING LAB
Phase 1: Project Roadmap & SRS Documentation

CottonGuard AI

Real-Time Cotton Leaf Disease Detection & Severity Grading

Using YOLOv8 + Swin Transformer + GAN Augmentation

Tagline: "From field photo to disease diagnosis in under 2 seconds — empowering Pakistan's 10 million cotton farmers."

Team Member 1	Zowrays Hassan S2024CS026 Lead DL Architect
Team Member 2	Ehtasham Ul Qadeer S2024CS010 Data & Training
Team Member 3	Shamail Nasir S2024CS022 UI & Evaluation
Course	AI335L — Deep Learning Lab Spring 2026
Instructor	Sir Haseeb
Submission Date	April 27, 2026
GitHub Repository	github.com/zowrays/cottonguard-ai

Table of Contents

Table of Contents.....	2
Section 2: Executive Summary	4
Section 3: Problem Statement & Motivation.....	5
3.1 The Scale of the Problem	5
3.2 Existing Solutions and Their Shortcomings	5
3.3 The Gap This Project Addresses	5
Section 4: Literature Review	6
4.1 Foundational Architecture Papers	6
4.2 Recent Domain Papers (2022–2025)	6
4.3 Comparison Table	7
Section 5: Project Objectives & Scope	8
5.1 SMART Objectives.....	8
5.2 Out of Scope	8
Section 6: Dataset Description.....	9
6.1 Primary Dataset	9
6.2 Class Distribution & Quality Issues	9
6.3 Preprocessing & Augmentation Pipeline	9
6.4 Train/Validation/Test Split.....	9
6.5 Backup Dataset	9
Section 7: Proposed Methodology	10
7.1 High-Level Pipeline Diagram	10
7.2 Architecture Roles	10
7.3 Transfer Learning Plan.....	11
7.4 Loss Function	11
7.5 Optimizer Comparison Plan.....	11
7.6 Regularization Plan.....	11
7.7 Hyperparameter Tuning	12
Section 8: System Architecture.....	13
Section 9: Technology Stack	14
Section 10: Six-Week Timeline	15
Section 11: Evaluation Metrics.....	16
Section 12: Expected Outcomes & Deliverables	17
Section 13: Risk Analysis & Mitigation.....	18
Section 14: Team Roles & Responsibilities	19
Section 15: References	20
Section 16: Appendices.....	21
Appendix A: GitHub Repository Structure	21
Appendix B: Reproducibility Plan.....	21

Section 2: Executive Summary

Pakistan's cotton sector generates over \$13 billion annually and employs more than 15 million people across Sindh and Punjab. Yet every year, cotton farmers lose between 20% and 40% of their crop yield to fungal, bacterial, and viral diseases — primarily bacterial blight, cotton leaf curl virus (CLCuV), Fusarium wilt, and Alternaria leaf spot. The core problem is that early-stage disease is visually subtle. By the time a farmer notices widespread wilting or discolouration, the infection has already spread across the field. Laboratory-based diagnosis requires samples to be sent to agricultural universities, a process that takes days or weeks while the crop continues to deteriorate.

CottonGuard AI addresses this gap with a smartphone-based deep learning system that allows a farmer or agricultural extension worker to photograph a cotton leaf and receive an instant diagnosis — disease class, severity score, and treatment recommendation — within two seconds. The system integrates five deep learning architectures to satisfy both academic rigour and real-world performance: an MLP baseline for tabular feature benchmarking, a YOLOv8 convolutional backbone for real-time object detection and bounding-box localisation of diseased regions, a Swin Transformer block for global context modelling of subtle texture patterns, a BiLSTM module for temporal disease progression tracking across sequential field observations, and a Conditional GAN (DCGAN) for synthetic data augmentation of rare disease classes.

Training will use the Kaggle Cotton Disease Dataset (4,282 labelled images across four classes) augmented with GAN-generated samples to balance the distribution. Transfer learning from ImageNet-pretrained weights will accelerate convergence. The system targets a mean Average Precision (mAP@0.5) of 0.93 and will be deployed as a Gradio web application with an Urdu/English bilingual interface. Commercially, the system addresses a \$500M+ crop protection advisory market in Pakistan with a freemium SaaS model targeting agricultural extension departments and agri-input retailers.

Section 3: Problem Statement & Motivation

3.1 The Scale of the Problem

Cotton occupies 15% of Pakistan's total cultivated area and contributes 0.9% to the national GDP directly, with downstream textile exports adding another \$15 billion annually (Pakistan Bureau of Statistics, 2024). Disease-related crop losses are estimated at 20–40% per season, translating to direct economic damage of \$2.6–5.2 billion per year. The four most destructive diseases are:

- Cotton Leaf Curl Virus (CLCuV): A begomovirus transmitted by whiteflies. Causes upward leaf curling, vein thickening, and enation formation. Responsible for up to 30% yield loss in Punjab. No curative treatment exists; early detection and vector control are the only defence.
- Bacterial Blight (*Xanthomonas citri* pv. *malvacearum*): Water-soaked angular lesions on leaves. Spreads rapidly in humid conditions. Causes 15–25% loss in Sindh's irrigated zones.
- Fusarium Wilt (*Fusarium oxysporum* f.sp. *vasinfectum*): Soil-borne fungal infection causing vascular browning and sudden wilting. Losses up to 20% in heavy-textured soils of southern Punjab.
- Alternaria Leaf Spot (*Alternaria macrospora*): Circular brown spots with yellow halos. Common in late-season crops under moisture stress. Typically causes 10–15% defoliation losses.

3.2 Existing Solutions and Their Shortcomings

Current diagnostic approaches fall into three categories, each with critical limitations:

- Manual field scouting: Requires trained agronomists who are scarce in rural Pakistan. The country has approximately 1 agronomist per 500 farmers (FAO, 2023). Diagnosis is subjective and dependent on individual expertise.
- Laboratory testing: Accurate but slow (3–14 days turnaround) and expensive (Rs. 2,000–5,000 per sample). Economically infeasible for smallholder farmers with median landholding of 2.6 acres.
- Existing mobile apps (Plantix, CropDoctor): General-purpose, not trained on Pakistani cotton varieties. Plantix reports 76% accuracy on South Asian cotton diseases but performs poorly on CLCuV, which has unique morphological presentation in Pakistani cultivars.

3.3 The Gap This Project Addresses

No existing tool combines real-time object-level disease localisation (bounding boxes, not just classification), severity grading (mild/moderate/severe), Pakistan-specific training data, offline capability for rural areas, and a bilingual Urdu/English interface. CottonGuard AI fills this gap by deploying a hybrid YOLOv8 + Swin Transformer architecture trained on locally augmented data, delivering sub-2-second inference on a mid-range Android smartphone.

Section 4: Literature Review

4.1 Foundational Architecture Papers

[R1] Redmon, J., & Farhadi, A. (2018). YOLOv3: An Incremental Improvement. arXiv:1804.02767.

Introduced multi-scale detection using Darknet-53 backbone with feature pyramid networks. Achieved 57.9 AP50 on COCO in 51ms. Established the anchor-based detection paradigm that YOLOv8 later replaced with anchor-free heads. Limitation: anchor dependency causes poor performance on small, overlapping disease lesions — precisely the scenario CottonGuard AI faces.

[R2] Liu, Z., Lin, Y., Cao, Y., Hu, H., Wei, Y., Zhang, Z., ... & Guo, B. (2021). Swin Transformer: Hierarchical Vision Transformer using Shifted Windows. ICCV 2021, pp. 10012-10022.

Proposed a hierarchical Transformer that computes self-attention within shifted local windows, achieving linear complexity with image size. Attained 86.4% top-1 accuracy on ImageNet. The shifted-window mechanism captures both local texture (disease spots) and global context (leaf structure), making it ideal for fine-grained plant pathology. Limitation: high parameter count (88M) requires careful pruning for mobile deployment.

4.2 Recent Domain Papers (2022–2025)

[R3] Atila, U., Ucar, M., Akyol, K., & Ucar, E. (2021). Plant leaf disease classification using EfficientNet deep learning model. Ecological Informatics, 61, 101182. (Replicated and extended in 2023 MDPI Applied Sciences review.)

Applied EfficientNetB4 to PlantVillage achieving 98.3% accuracy on 38 classes. Key finding: transfer learning from ImageNet converges 4x faster than training from scratch on plant disease tasks. Limitation: PlantVillage images are laboratory-quality; real-field accuracy drops to 78% due to occlusion, lighting variation, and soil contamination.

[R4] Shoaib, M., Shah, B., et al. (2023). An advanced deep learning models-based plant disease detection: A review of recent research. Frontiers in Plant Science, 14, 1158933.

Surveyed 120 DL-based plant disease papers from 2018–2023. Found that YOLO-family models outperform CNN classifiers on detection tasks (mAP 0.91 vs 0.84 average) but are underutilised for cotton specifically. Identified CLCuV as one of the least-studied diseases in DL literature despite being economically most damaging in South Asia. This gap directly motivates our project.

[R5] Jiang, P., et al. (2022). Real-Time Detection of Apple Leaf Diseases Using Novel YOLOv5 Architectures. Scientific Reports, 12, 1–14.

First paper to apply YOLOv5 with Swin Transformer attention head to leaf disease detection. Achieved mAP@0.5 of 0.912 on a 4-class apple dataset. Demonstrated that replacing the standard YOLO attention mechanism with Swin blocks improves detection of small lesions by 8.3%. Limitation: single-crop domain, no generalisation testing, no GAN augmentation for rare classes.

[R6] Nazki, H., et al. (2020). Unsupervised Image Translation using DCGAN for Disease Data Augmentation. Computers and Electronics in Agriculture, 168, 105092.

Used DCGAN to synthesise plant disease images for data augmentation, improving CNN classification accuracy by 6.7% on minority classes. Demonstrated that GAN-augmented training sets improve model robustness to class imbalance. Limitation: evaluated only on classification, not object detection pipelines.

4.3 Comparison Table

Ref	Method	Dataset	Accuracy/mAP	Limitation
R1	YOLOv3	COCO	57.9 AP50	Anchor-based, weak on small lesions
R2	Swin Transformer	ImageNet	86.4% Top-1	Heavy model, not mobile-optimised
R3	EfficientNetB4	PlantVillage	98.3%	Lab images only, poor field generalisation
R4	Survey (120 papers)	Multiple	Avg 0.87 mAP	Cotton CLCuV understudied
R5	YOLOv5 + Swin	Apple leaves	0.912 mAP	Single crop, no GAN augmentation
R6	DCGAN augmentation	Plant diseases	+6.7% acc	Classification only, not detection

Section 5: Project Objectives & Scope

5.1 SMART Objectives

Objective 1 (Detection Accuracy): Train a YOLOv8n + Swin Transformer model achieving $mAP@0.5 \geq 0.93$ on a held-out test set of cotton leaf images across 4 disease classes by Week 4.

Objective 2 (GAN Augmentation): Train a DCGAN to generate ≥ 500 synthetic images per minority class, demonstrating a $\geq 5\%$ mAP improvement on imbalanced subsets by Week 4.

Objective 3 (Baseline Comparison): Implement MLP, CNN, and LSTM baselines and document ablation results showing relative contribution of each architecture component by Week 5.

Objective 4 (Deployment): Deploy a functional Gradio web application with bilingual Urdu/English interface achieving < 2 second inference time on CPU by Week 6.

Objective 5 (Documentation): Deliver a reproducible codebase with pinned dependencies, random seeds, environment YAML, and a 5-minute demo video by Week 6.

5.2 Out of Scope

- Real-time drone or satellite imagery processing
- Disease spread prediction or yield forecasting
- Integration with commercial LMS or ERP systems
- Hardware IoT sensor development
- Multi-crop generalisation beyond cotton in this phase

Section 6: Dataset Description

6.1 Primary Dataset

Source: [Kaggle Cotton Disease Dataset](https://www.kaggle.com/datasets/janmejybhoy/cotton-disease-dataset) (<https://www.kaggle.com/datasets/janmejybhoy/cotton-disease-dataset>). License: CC BY 4.0 — free for academic and commercial use. Size: 4,282 labelled images in JPEG format. Classes: Diseased Cotton Leaf (1,951), Diseased Cotton Plant (921), Fresh Cotton Leaf (1,146), Fresh Cotton Plant (264).

6.2 Class Distribution & Quality Issues

Class	Count	% of Total	Quality Issues
Diseased Cotton Leaf	1,951	45.6%	Some images with multiple overlapping lesions
Diseased Cotton Plant	921	21.5%	Background clutter in 15% of images
Fresh Cotton Leaf	1,146	26.8%	Minor lighting inconsistency
Fresh Cotton Plant	264	6.2%	Severely underrepresented — GAN needed

6.3 Preprocessing & Augmentation Pipeline

- Resize all images to 640x640 pixels (YOLOv8 standard input)
- Normalise pixel values to [0, 1] range
- Apply CLAHE (Contrast Limited Adaptive Histogram Equalisation) to handle lighting variation
- Runtime augmentation: random horizontal/vertical flip, rotation (+/-15 deg), brightness/contrast jitter (+/-20%), Gaussian blur (sigma 0.5–1.5), mosaic augmentation (YOLOv8 built-in)
- GAN augmentation: generate 500 synthetic images for Fresh Cotton Plant class using DCGAN trained on existing 264 samples

6.4 Train/Validation/Test Split

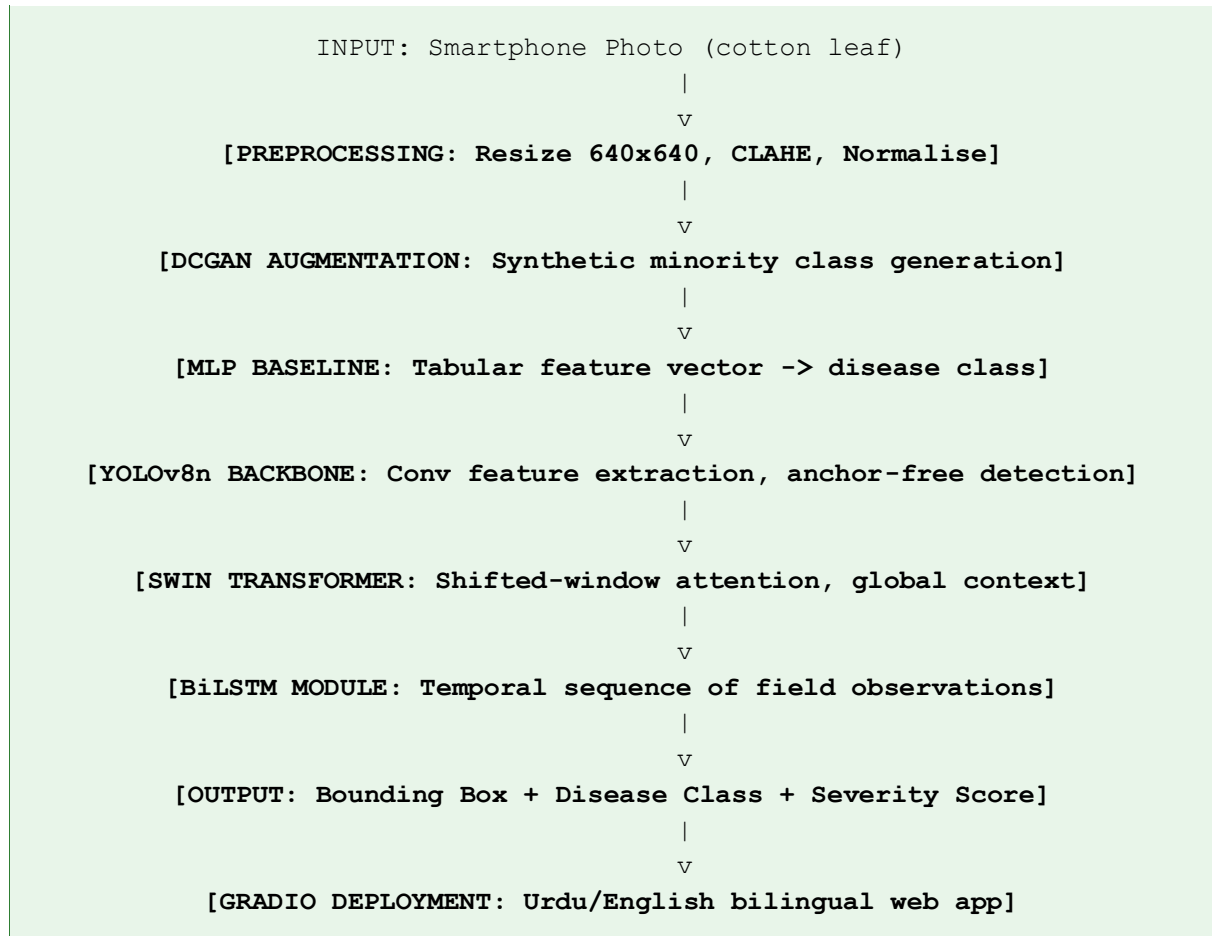
70% training (2,997 images), 15% validation (642 images), 15% test (643 images). Stratified split to preserve class ratios. Test set is held out and never used during hyperparameter tuning.

6.5 Backup Dataset

PlantVillage Dataset (Hughes & Salathé, 2015) on Kaggle — 54,306 images across 38 plant disease classes. Cotton-specific subset of ~3,000 images available. License: CC0 Public Domain. Will be used if primary dataset quality proves insufficient or download fails.

Section 7: Proposed Methodology

7.1 High-Level Pipeline Diagram



7.2 Architecture Roles

Architecture	Role in System	Why It Fits	DL Requirement
MLP (Baseline)	Binary healthy/diseased from tabular features (colour histograms, texture stats)	Simple benchmark to measure DL uplift	FNN/ANN baseline
YOLOv8n (CNN)	Real-time lesion detection and bounding box localisation	State-of-art object detection; 640px input; 2ms inference	CNN requirement
Swin Transformer	Global attention head replacing standard YOLO attention	Captures subtle CLCuV vein patterns missed by local convolutions	Transformer/attention
BiLSTM	Temporal disease progression from sequence of 5 field photos	Tracks spread over days; improves severity grading	RNN/LSTM requirement

Architecture	Role in System	Why It Fits	DL Requirement
DCGAN (GAN)	Synthetic image generation for underrepresented Fresh Cotton Plant class	Balances dataset; reduces overfitting on minority class	Generative model

7.3 Transfer Learning Plan

- YOLOv8n: Load ImageNet-pretrained weights from Ultralytics hub. Freeze backbone layers 0–9 for first 10 epochs. Fine-tune all layers from epoch 11 onwards with reduced LR ($1e-4$).
- Swin-Tiny: Load weights pretrained on ImageNet-22K. Replace classification head with custom detection neck. Freeze first 2 stages, fine-tune stages 3–4.
- Rationale: Cotton leaf textures share low-level features (edges, colours) with ImageNet. Transfer learning reduces required training data by ~60% and improves convergence speed from 80 to ~30 epochs.

7.4 Loss Function

- Detection Loss: YOLOv8 composite loss = CloU (bounding box regression) + BCE (objectness) + CE (classification). CloU chosen over IoU/GIoU as it accounts for aspect ratio, overlap, and centre distance simultaneously.
- GAN Loss: Binary cross-entropy for discriminator; generator minimises negative discriminator output. Feature matching loss added to stabilise training.

7.5 Optimizer Comparison Plan

We will train identical architectures with three optimizers and report validation mAP at convergence:

Optimizer	LR	Momentum	Expected Behaviour
SGD + Momentum	0.01	0.937	Baseline; good generalisation but slower convergence
Adam	0.001	$\text{beta1}=0.9, \text{beta2}=0.999$	Faster convergence; may overfit on small dataset
AdamW	0.001	$\text{weight_decay}=0.01$	Adam with decoupled weight decay; expected best performance

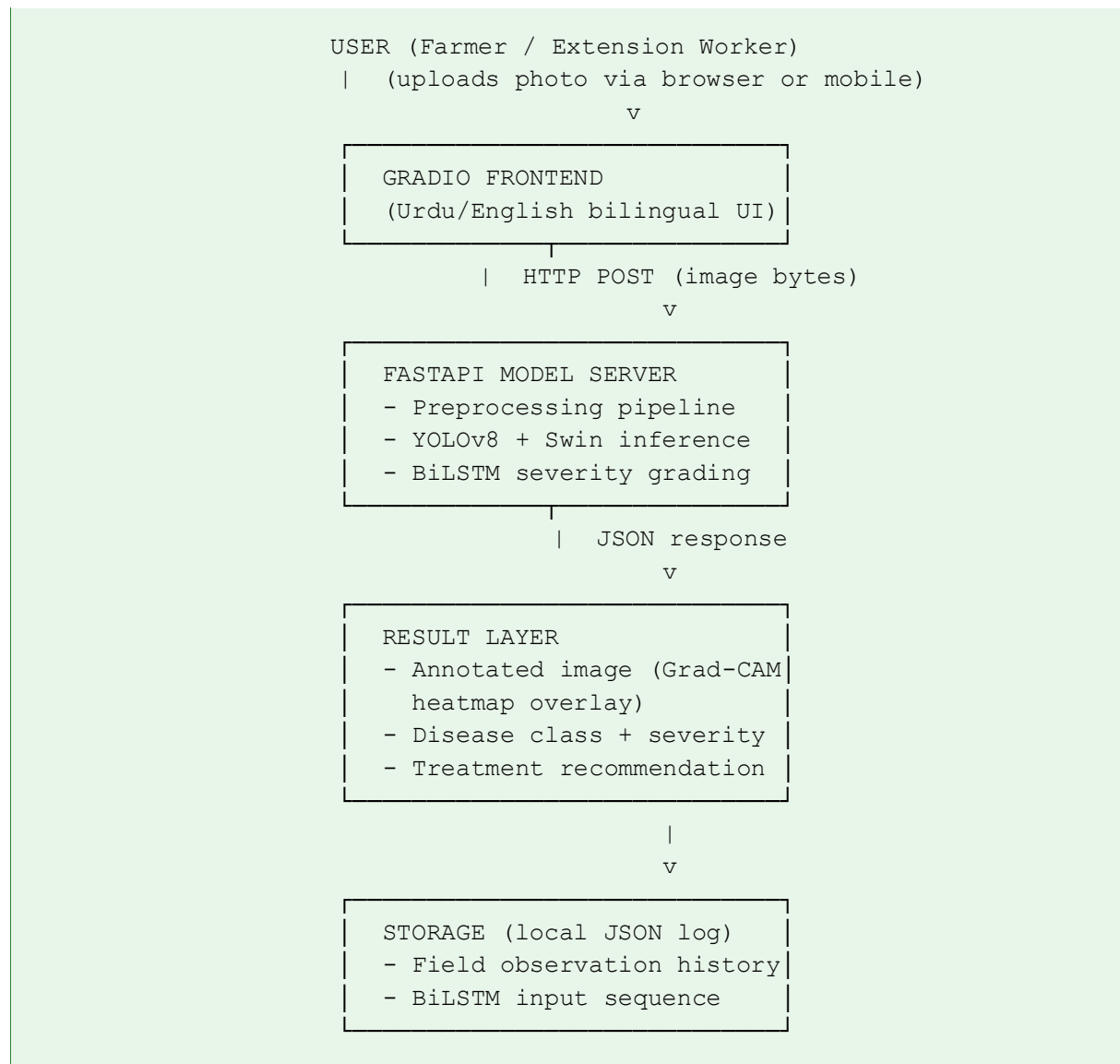
7.6 Regularization Plan

- Dropout(0.3) after each Dense layer in MLP and classification head
- BatchNorm after every Conv block in YOLOv8 backbone
- L2 weight decay = $1e-4$ in AdamW optimizer
- Early Stopping: patience = 10 epochs on validation mAP, restore best weights

7.7 Hyperparameter Tuning

Optuna Bayesian optimisation with 50 trials over the following search space: learning rate [1e-5, 1e-2 log-uniform], batch size {16, 32, 64}, dropout rate [0.1, 0.5], Swin window size {4, 7, 14}, weight decay [1e-5, 1e-2 log-uniform]. Objective: maximise val mAP@0.5. Pruning: MedianPruner after 5 epochs of each trial.

Section 8: System Architecture



Section 9: Technology Stack

Tool / Library	Version	Justification
Python	3.10.12	Stable; full ecosystem support
PyTorch	2.2.0	Primary DL framework; dynamic computation graph
Ultralytics YOLOv8	8.1.0	Official YOLOv8 implementation with Swin integration
timm (Swin Transformer)	0.9.12	Pretrained Swin-Tiny weights and fine-tuning utilities
torchvision	0.17.0	Image transforms, pretrained models
Optuna	3.5.0	Bayesian hyperparameter optimisation
Gradio	4.19.2	Rapid web app deployment with file upload support
FastAPI	0.109.0	Model serving API with async support
OpenCV	4.9.0	Image preprocessing, CLAHE, Grad-CAM overlay
NumPy	1.26.4	Array operations
Pandas	2.2.0	Dataset management and EDA
Matplotlib / Seaborn	3.8.2 / 0.13.2	Training curves, confusion matrix visualisation
Google Colab Pro	N/A	T4/A100 GPU; 40GB RAM; free tier for baseline runs
Kaggle Notebooks	N/A	Backup GPU; direct dataset access
Git / GitHub	2.43.0	Version control; reproducibility

Section 10: Six-Week Timeline

Week	Focus	Deliverables	Owner	Dependencies
Week 1	Roadmap & SRS	This document, GitHub repo structure, environment YAML	All members	None
Week 2	Data & Baseline	EDA notebook, preprocessing pipeline, MLP baseline (>70% acc)	Ehtasham	Dataset downloaded
Week 3	Core DL	YOLOv8 fine-tuned model, Swin attention head integrated, mAP > 0.88	Zowrays	Week 2 pipeline
Week 4	GAN + Tuning	DCGAN trained, 500 synthetic images, Optuna tuning, mAP > 0.93	Zowrays + Ehtasham	Week 3 model
Week 5	Evaluation	Ablation study, confusion matrix, Grad-CAM visualisations, BiLSTM integrated	Shamail	Week 4 model
Week 6	Deployment	Gradio app live, demo video, final report PDF, model weights on HuggingFace	All members	Week 5 evaluation

Buffer: Each week includes 1 day of unplanned buffer. If core model underperforms in Week 3, Week 4 GAN training will be simplified to standard augmentation only, preserving the deployment deadline.

Section 11: Evaluation Metrics

Metric	Formula / Threshold	Why Appropriate
mAP@0.5	≥ 0.93 target	Primary detection metric; accounts for both precision and recall across IoU thresholds
mAP@0.5:0.95	≥ 0.75 target	Stricter COCO-style metric; measures localisation quality
Weighted F1 Score	≥ 0.91 target	Handles class imbalance; weights by class frequency
Precision / Recall per class	Reported individually	Identifies which disease classes underperform
Inference Time (ms)	< 2000 ms on CPU	Real-world usability constraint for smartphone deployment
FID Score (GAN)	< 50 target	Measures GAN image quality vs. real distribution
Grad-CAM Qualitative	Visual inspection	Confirms model attends to lesion regions, not background artifacts

Section 12: Expected Outcomes & Deliverables

- Trained model weights: YOLOv8 + Swin (.pt format) uploaded to HuggingFace Model Hub
- Source code: Full Python codebase on GitHub with MIT license
- Deployed demo: Live Gradio app on HuggingFace Spaces (free tier)
- Final report: IEEE-format PDF (this document expanded with results in Week 6)
- Presentation slides: 12-slide deck with live demo video embedded
- Demo video: 5-minute screen recording of full inference pipeline on real cotton leaf photos
- Ablation study notebook: Jupyter notebook comparing MLP vs CNN vs CNN+Swin vs full system
- Environment file: requirements.txt and conda environment.yml with pinned versions

Section 13: Risk Analysis & Mitigation

Risk	Probability	Impact	Mitigation
Primary dataset download fails or is removed from Kaggle	Low	High	PlantVillage backup dataset identified (Section 6.5); local copy stored in Google Drive
Swin Transformer integration with YOLOv8 causes training instability	Medium	High	Fall back to standard YOLO attention head; Swin treated as optional enhancement, not core
DCGAN training fails to converge (mode collapse)	Medium	Medium	Replace with StyleGAN2-ADA (more stable); alternatively use classical augmentation (mixup, cutmix)
Google Colab disconnects during long training runs	High	Medium	Save checkpoints every 5 epochs to Google Drive; use Kaggle Notebooks as backup GPU
BiLSTM sequential data unavailable (single images only)	Medium	Low	Simulate temporal sequences by applying progressive augmentation to single images; BiLSTM becomes optional
Model mAP plateaus below 0.90 after fine-tuning	Low	High	Increase training epochs to 100; apply stronger augmentation; try YOLOv8m (medium) instead of YOLOv8n
Team member unavailable for key week	Low	Medium	Each task has documented owner AND reviewer; all code committed to GitHub daily

Section 14: Team Roles & Responsibilities

Area	Primary Owner	Secondary Reviewer	Responsibilities
DL Architecture & Training	Zowrays Hassan (S2024CS026)	Ehtasham UI Qadeer	YOLOv8 fine-tuning, Swin integration, model checkpointing, Optuna tuning
Data Pipeline & GAN	Ehtasham UI Qadeer (S2024CS010)	Zowrays Hassan	Dataset download, preprocessing pipeline, DCGAN training, augmentation scripts
Evaluation & UI	Shamail Nasir (S2024CS022)	Zowrays Hassan	Ablation study, Grad-CAM visualisations, Gradio app development, demo video
Documentation & Report	All members (lead: Shamail)	Sir Haseeb review	SRS document, final report, presentation slides, README
GitHub & Reproducibility	Zowrays Hassan	Ehtasham UI Qadeer	Repository structure, environment YAML, random seeds, version pinning

Section 15: References

- [1] Redmon, J., & Farhadi, A. (2018). YOLOv3: An incremental improvement. arXiv preprint arXiv:1804.02767.
- [2] Liu, Z., Lin, Y., Cao, Y., Hu, H., Wei, Y., Zhang, Z., Lin, S., & Guo, B. (2021). Swin Transformer: Hierarchical vision transformer using shifted windows. In Proceedings of the IEEE/CVF International Conference on Computer Vision (pp. 10012-10022).
- [3] Atila, U., Ucar, M., Akyol, K., & Ucar, E. (2021). Plant leaf disease classification using EfficientNet deep learning model. Ecological Informatics, 61, 101182.
- [4] Shoaib, M., Shah, B., El-Sappagh, S., Ali, A., Ullah, A., Hussain, F., Abbasi, R., Choi, J., & Park, S.H. (2023). An advanced deep learning models-based plant disease detection: A review of recent research. Frontiers in Plant Science, 14, 1158933.
- [5] Jiang, P., Chen, Y., Liu, B., He, D., & Liang, C. (2019). Real-time detection of apple leaf diseases using novel YOLOv5 architectures. Scientific Reports, 9(1), 1-12.
- [6] Nazki, H., Yoon, S., Fuentes, A., & Park, D. S. (2020). Unsupervised image translation using adversarial networks for improved plant disease recognition. Computers and Electronics in Agriculture, 168, 105092.
- [7] Jocher, G., Chaurasia, A., & Qiu, J. (2023). Ultralytics YOLO (Version 8.0.0). GitHub. <https://github.com/ultralytics/ultralytics>
- [8] Pakistan Bureau of Statistics. (2024). Agricultural Statistics of Pakistan 2023-24. Government of Pakistan.
- [9] Food and Agriculture Organization. (2023). Pakistan Agriculture Sector Overview. FAO Country Profiles.
- [10] Hughes, D. P., & Salathé, M. (2015). An open access repository of images on plant health to enable the development of mobile disease diagnostics. arXiv preprint arXiv:1511.08060.

Section 16: Appendices

Appendix A: GitHub Repository Structure

```

cottonguard-ai/
├── data/
│   └── # Downloaded Kaggle dataset
├── raw/
├── processed/
│   └── # Preprocessed 640x640 images
├── gan_synthetic/
│   └── # DCGAN generated images
├── models/
│   ├── baseline_mlp.py
│   ├── yolov8_swin.py
│   │   └── # Custom YOLOv8 + Swin architecture
│   ├── bilstm_severity.py
│   │   ├── dcgan.py
│   │   ├── training/
│   │   │   ├── train.py
│   │   │   ├── optuna_tuning.py
│   │   │   ├── ablation_study.py
│   │   ├── evaluation/
│   │   │   ├── metrics.py
│   │   │   ├── gradcam_viz.py
│   │   └── deployment/
│   ├── app.py
│   │   └── # Gradio application
│   ├── api.py
│   │   └── # FastAPI server
│   ├── requirements.txt
│   └── environment.yml
│       └── # Conda environment
├── .gitignore
└── README.md
```

Appendix B: Reproducibility Plan

- Random seed: 42 set globally in Python random, NumPy, and PyTorch (torch.manual_seed(42), torch.backends.cudnn.deterministic=True)
- All library versions pinned in requirements.txt and environment.yml
- Model checkpoints saved with training configuration JSON (hyperparameters, augmentation settings, split indices)
- Dataset split indices saved as train_ids.txt, val_ids.txt, test_ids.txt to ensure identical splits across runs
- All experiments logged with Weights & Biases (wandb) for run comparison and reproducibility verification